Proyecto Final — Aplicación Flutter de tema libre

Curso: Desarrolla Aplicaciones Móviles Robustas con Flutter e IA

Peso en la nota: 40 %

Entrega: en Moodle, **hasta el lunes 26 de octubre a las 8:00 a. m.**

> Este documento reúne todo lo que necesitas para el Proyecto Final: qué construir, qué subir a Moodle, cómo se evalúa y cómo cerrar la entrega. Es el mismo contenido que se explicó en la Sesión 10. **No hay presentación ni demostración en vivo:** quien evalúa revisa tu repositorio, ejecuta tu APK y sigue tu README.

1. Qué es el Proyecto Final

A lo largo del curso construiste *ExploraEC*, sesión a sesión, junto con el resto del grupo. Es el proyecto de práctica guiada, **no** lo que entregas al final.

El Proyecto Final es una aplicación Flutter **aparte**, de **tema libre**, construida por tu cuenta. Aplicas el mismo stack y los mismos patrones practicados: autenticación, datos remotos, persistencia local, una funcionalidad remota o colaborativa, un asistente de IA a través de un servidor propio, estado centralizado con GetX y arquitectura en capas. La diferencia es que lo haces sobre un dominio que tú eliges. No tiene que tratarse de lugares ni parecerse a ExploraEC.

No necesita ser un tema complicado ni una app grande. Algunos ejemplos válidos:

- un registro de gastos personales,
- una lista de tareas colaborativa,
- un catálogo de recetas con favoritos,
- un rastreador de hábitos,
- una app de reseñas de libros o películas.

Lo que importa es que cubra la superficie técnica de las secciones 3 y 4, no el tamaño del dominio.

Objetivo: entregar una aplicación Flutter propia, completa, funcional y documentada, que demuestre sobre un dominio distinto al de ExploraEC los mismos resultados de aprendizaje

practicados en las 10 sesiones.

2. Elegir el tema

1. Elige un dominio que te interese y que tenga sentido con los requisitos de abajo. Necesitas una fuente de datos real o, al menos, un modelo de datos propio con varios registros.
2. Evita un dominio que dependa de una API de pago, con cuota gratuita insuficiente para desarrollar, o que requiera datos sensibles de terceros.
3. Si tienes dudas sobre si tu tema cumple los requisitos mínimos, consúltalas con el docente. **No es obligatorio** consultarle ni pedirle confirmación del tema: la decisión es tuya.

3. Requisitos funcionales mínimos

1. **Registro e inicio de sesión** de usuario, con autenticación y persistencia remota. Puede ser un backend propio (por ejemplo, con JWT) o Firebase Authentication; ambas opciones son válidas. También puedes reutilizar el backend de gastos del curso (https://github.com/cj-murillo/proyecto-curso-spec-kit), sabiendo que solo ofrece usuarios y la entidad *gasto*.
2. **Una lista de datos de tu dominio**, con estados de carga, vacío y error explícitos. Los datos deben venir de una fuente real (una API pública o un conjunto de datos abierto) o de un conjunto propio en tu backend, Firestore o Hive con **al menos 15-20 registros**.
3. **Persistencia local** de al menos una funcionalidad (favoritos, historial, borradores o equivalente) que sobreviva al reinicio de la app.
4. **Una funcionalidad de datos remotos del usuario o colaborativa** (comentarios, calificaciones, una lista compartida, un CRUD de tu entidad o equivalente), guardada en tu backend o en Firestore y protegida por autorización real (token/JWT o reglas de seguridad). Si es colaborativa, debe ser visible para varios usuarios.
5. **Una funcionalidad de asistencia con IA** relevante a tu dominio (recomendaciones, resúmenes, clasificación o respuestas sobre tus datos), consumida a través de un **servidor propio**. Nunca una clave de un proveedor de modelos dentro de la app.
6. **Gestión de estado centralizada con GetX**, sin `setState` disperso para el estado compartido entre pantallas.

Un mapa o la geolocalización **no** son un requisito: solo tienen sentido si tu tema lo amerita.

4. Requisitos técnicos y de experiencia de usuario

Técnicos

- Flutter y Dart con las versiones que usaste durante el curso.
- Arquitectura en capas: pantalla (`screens/`) → controlador (GetX) → servicio o repositorio → fuente de datos. Ninguna pantalla hace una llamada HTTP o a Firestore directamente, y el token se guarda con almacenamiento seguro.
- Sin claves ni credenciales reales en el repositorio: ni `.env`, ni claves secretas, ni tokens en los registros. Incluye un `.env.example` sin valores reales.
- Repositorio Git con **historial real de commits**, no un único commit final.

De experiencia de usuario

- Los estados de carga, vacío y error son visibles y distintos entre sí. Ninguna pantalla queda en blanco mientras espera datos.
- Ninguna pantalla es un callejón sin salida.
- Hay consistencia visual (colores, tipografía y espaciado) en toda la app.

Extensiones opcionales (no son obligatorias): mapa o geolocalización si tu dominio tiene una dimensión geográfica real, notificaciones locales, modo oscuro o cualquier funcionalidad adicional que enriquezca tu tema.

5. Qué se sube a Moodle

En la tarea de Moodle se sube **una sola cosa: el enlace a tu repositorio** de GitHub (público, o privado con acceso para el docente), con historial de commits real.

Todo lo demás se revisa **dentro del repositorio**, por eso también es parte de la entrega:

- **El Release con el APK.** No se sube a Moodle: lo genera GitHub Actions al subir una etiqueta (por ejemplo, `v1.0.0`), con el workflow que se practicó en la Sesión 10, y queda publicado en la sección *Releases* de tu repositorio. El APK debe poder instalarse en un emulador Android.
- **El `README.md`.** GitHub lo muestra en la página principal del repositorio; no hay que subirlo aparte.

6. Cómo se evalúa

La rúbrica tiene **nueve criterios** y es la misma sin importar el tema que elijas; solo cambia qué cuenta como "fuente real de datos" o "funcionalidad colaborativa" según tu dominio. Cada criterio se comprueba con **evidencia observable**: algo que quien evalúa puede ver ejecutando tu APK, siguiendo tu README o señalando en el repositorio.

Criterio	Evidencia observable	Peso
Persistencia remota o colaborativa	Se puede crear, ver y eliminar contenido remoto (CRUD de la entidad, comentarios, calificaciones, lista compartida o equivalente), protegido por autorización real. Si es colaborativo, los cambios se reflejan para otro usuario	20 %
Autenticación	Registro, inicio y cierre de sesión funcionan, con el token en almacenamiento seguro. Una funcionalidad protegida rechaza a un usuario no autenticado	15 %
Datos reales	La lista principal viene de una fuente real o de un conjunto propio de al menos 15-20 registros en tu backend, Firestore o Hive; no está escrita a mano en el código	15 %
Persistencia local	Una funcionalidad guardada sobrevive al cierre completo de la app	10 %
Estado con GetX	El estado compartido entre pantallas vive en al menos un `GetxController`, no replicado con `setState` en cada pantalla	10 %
IA sin secretos	El asistente llama a un servidor propio; una búsqueda de patrones de claves en el repositorio no encuentra nada real y `.env` no está versionado	10 %
Manejo de errores	Cada pantalla con datos cubre carga, vacío y error, comprobado forzando al menos un error (apagar el backend o quitar la red; con un backend HTTP, mensajes distintos para 401, 400, 422 y 5xx)	10 %
Arquitectura	Separación real entre pantalla, estado y servicios: ninguna pantalla llama directo a HTTP o Firestore	5 %
Documentación y entrega	README completo que permite ejecutar el proyecto sin preguntar nada; APK instalable en un Release de GitHub generado con GitHub Actions; repositorio con historial real de commits	5 %

Total: 100 %.

Para tener en cuenta:

- **Encontrar una clave real en cualquier parte del repositorio** resta puntos adicionales, a criterio del docente, sin importar la funcionalidad. Es una falta de seguridad real, no un detalle de estilo.
- Las **extensiones opcionales** no tienen fila propia: no restan si faltan, pero si las incluyes y están bien implementadas pueden sumar hasta un punto extra a criterio del docente.
- La funcionalidad y la experiencia de usuario son criterios independientes: una app que cumple todo pero tiene una experiencia pobre puede aprobar con nota media.

7. El README

Es parte de la entrega, pero se revisa en el repositorio, no en Moodle. Puedes partir de la plantilla `docs/README-plantilla.md` (ver la sección 8). Debe cumplir tres propósitos:

Para…	Qué incluye
Ejecutar	Qué hace la app, capturas de pantalla, requisitos, cómo levantar el backend con los comandos exactos, las direcciones por entorno y cómo instalar el APK
Entender	Diagrama de capas y **una decisión técnica propia** (por qué elegiste algo y qué descartaste)
Confiar	Seguridad (dónde viven las claves y el token), créditos y licencias de las fuentes y los paquetes

La documentación técnica breve puede vivir en el mismo `README.md` o en `docs/ARQUITECTURA.md`.

Sugerencia: si puedes, pídele a alguien que lo siga en **otro computador**. Si tiene que preguntarte algo, probablemente el README está incompleto. Si trabajas solo, sigue tú mismo los pasos en un entorno limpio (otra carpeta o un emulador nuevo).

8. El plan de la semana

Una semana alcanza para **cerrar bien**, no para construir desde cero. Conviene avanzar el proyecto en paralelo a las sesiones del curso y dejar la semana final para el cierre. El release va en los últimos días, con margen por si algo falla.

Días	Qué hacer
1-3	Revisar que tu proyecto cumpla los requisitos y corregir lo que falte. Llevar a tu proyecto el flujo de CI y release de la Sesión 10 (la carpeta `.github`) y comprobar que la CI se pone en verde. Si usas un agente de código, añade también `AGENTS.md`, `CLAUDE.md` y `.claude`.
3-5	Escribir el README y hacer la auditoría de secretos. Hacer commits durante la semana, no uno solo al final.
6-7	Publicar el release (`git tag v1.0.0` y `git push origin v1.0.0`), probar el APK en un emulador limpio y subir el enlace del repositorio a Moodle **antes** de la hora límite.

Archivos de apoyo (opcionales). Si quieres una guía, puedes tomar como ejemplo estos archivos de la rama `sesion-10` del repositorio de ExploraEC (https://github.com/Patricio-CEDIA/exploraec-app/tree/sesion-10). Son una sugerencia: adáptalos a tu proyecto o hazlos a tu manera.

Archivo	Para qué sirve
`docs/README-plantilla.md`	Punto de partida para el README
`docs/lista-de-verificacion.md`	Lista para revisar tu proyecto antes de publicar
`.github/workflows/`	Flujos de CI y de publicación del APK
`AGENTS.md`, `CLAUDE.md`, `.claude/`	Instrucciones para agentes de código

9. Antes de subir: lista corta

- ☐ La **CI** (`.github/workflows/ci.yml`) está en verde en tu rama principal.
- ☐ El **Release** existe, tiene el APK adjunto y lo instalaste en un emulador limpio.
- ☐ La **auditoría de secretos** salió limpia: ninguna clave, `.env` ignorado, sin tokens en los registros.
- ☐ Revisaste tu proyecto contra los requisitos de las secciones 3 y 4 y corregiste lo que faltaba.
- ☐ El `README.md` se puede seguir **sin preguntarte nada** (ideal: probado por otra persona o en un entorno limpio).
- ☐ Si usaste un agente de código, leíste y verificaste lo que escribió, y tu `AGENTS.md`

describe tu proyecto (no el de ExploraEC).

- ☐ Subiste a Moodle **el enlace del repositorio**, antes del lunes 26 a las 8:00 a. m.

10. Errores frecuentes en las entregas

Error	Qué pasa	Cómo evitarlo
README que solo funciona en tu computador	Quien evalúa no puede ejecutarla	Seguirlo tú en un entorno limpio
Clave o archivo de entorno en el repositorio	Resta puntos y es un riesgo real	Auditar antes de publicar
APK que no se instala o no arranca	No se puede comprobar nada	Probarlo en un emulador limpio
Un solo commit al final	No hay historial de trabajo	Hacer commits durante la semana
Release dejado para el último día	Si falla, no hay tiempo	Publicarlo con margen
CI en rojo en la entrega	Muestra que lo entregado no pasa sus propias pruebas	Mirar la pestaña Actions antes de subir
Código de un agente que nadie leyó	Errores, secretos o dependencias de más	Leer el `git diff` y verificar con `flutter analyze` y `flutter test`

11. Si algo falla en GitHub Actions

- **No aparece ninguna ejecución.** El archivo del workflow no estaba en el commit que subiste, o la etiqueta se subió antes que el workflow. Sube el commit y crea la etiqueta de nuevo con otro nombre (`v1.0.1`).
- **`Resource not accessible by integration` al publicar el Release.** En GitHub: *Settings* → *Actions* → *General* → *Workflow permissions* → *Read and write permissions*, y vuelve a ejecutar.
- **Una ejecución se pone en rojo.** Abre la ejecución en la pestaña Actions, mira qué paso falló y reprodúcelo con el mismo comando en tu computador. No desactives la prueba para ponerlo en verde.

- **El APK no se instala sobre el de depuración.** Están firmados con llaves distintas: ejecuta `adb uninstall <id-de-tu-app>` antes de instalar el del Release.

12. Relación con ExploraEC

ExploraEC es la referencia de cómo se ven estos patrones implementados. Puedes consultarla como ejemplo, pero el Proyecto Final es tu propio código, sobre tu propio tema: no una copia ni una variación de ExploraEC.